

Convolutional Neural Networks

Understanding Deep Learning

Firuz Kamalov

MAI 604: Deep Learning

April 30, 2026

Lecture Outline

- 1 Motivation & Background
- 2 1D Convolutional Networks
- 3 2D Convolutional Networks
- 4 Downsampling and Upsampling
- 5 Applications
- 6 Summary

Processing Image Data

- Previously, we considered **fully connected** neural networks where every input connects to every hidden unit.
- Applying this to image data poses severe problems:
 - ① **High Dimensionality:** A typical 224×224 RGB image has 150,528 dimensions. A fully connected hidden layer of similar size requires ~ 22 billion weights!
 - ② **Spatial Relationships:** Fully connected layers treat all inputs equally, ignoring that nearby pixels are strongly correlated.
 - ③ **Geometric Transformations:** A fully connected model must relearn what a tree looks like at every position in the image.
- **Solution:** Convolutional Neural Networks (CNNs). They process local image regions independently using *shared* parameters.

Invariance and Equivariance

- **Translation Invariance:** a neural network can recognize an object regardless of where it is located.
- A cat in the top-left is recognized just as easily as one in the bottom-right.
- Before CNNs, MLPs struggled terribly with this. If trained on cats perfectly centered, shifting the cat just 20 pixels would make the MLP completely fail because it memorized exact pixel locations.

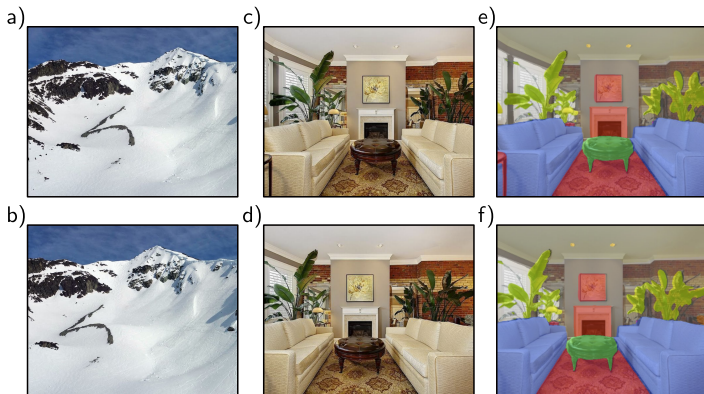


Figure 1: Invariance (a-b) vs Equivariance (c-f) for translation.

1D Convolution Operation

- 1D convolution moves in one direction (vertical).
- Each output z_i is a weighted sum of nearby inputs using a shared **convolution kernel** (filter) ω .
- E.g., for kernel size 3:

$$z_i = \omega_1 x_{i-1} + \omega_2 x_i + \omega_3 x_{i+1}$$

- **Padding:** Handling boundary outputs.
 - *Zero-padding:* Add zeros at the ends of the sequence ($L_{out} = L_{in}$).
 - *Valid conv:* No padding ($L_{out} < L_{in}$).

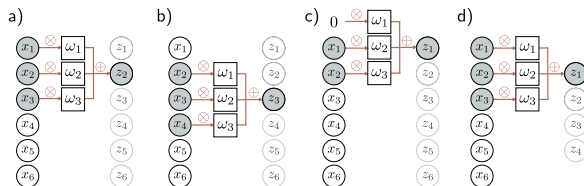


Figure 2: 1D convolution with kernel size three. $\mathbf{z} = \mathbf{x} * \omega$

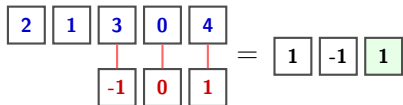
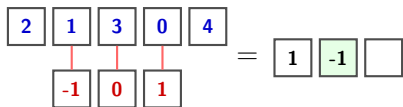
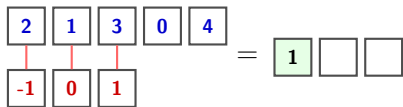
Numeric Example: 1D Convolution

Input Vector: $x = [2, 1, 3, 0, 4]$

Kernel: $\omega = [-1, 0, 1]$

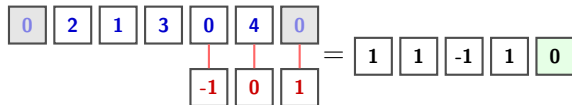
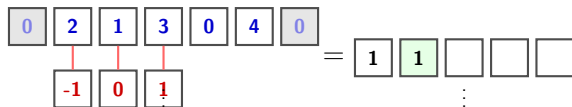
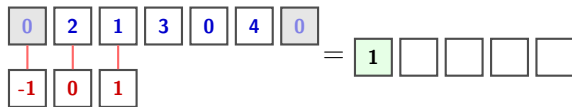
Case 1: Valid Convolution

(No Padding) $\rightarrow$ Output length 3



Case 2: Zero-Padding

(Pad = 1) $\rightarrow$ Output length 5



Stride, Kernel Size, and Dilation

Convolutional families are defined by three parameters:

- **Stride:** How many positions the kernel shifts at each step. $\text{Stride} > 1$ reduces spatial dimensions.
- **Kernel Size:** Size of the filter (usually odd).
- **Dilation:** Insert gaps between the filter values. Allows integration over a larger area without increasing parameter count.

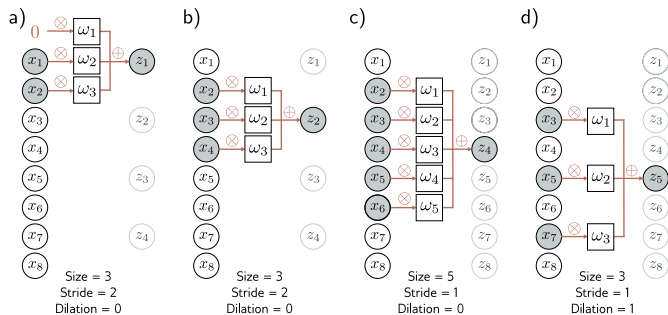


Figure 3: Modifying Stride, Kernel Size, and Dilation.

Convolutional vs. Fully Connected Layers

- A convolutional layer is just a special case of a fully connected layer!
- Convolution enforces sparsity (most weights are zero) and weight sharing (weights repeat).

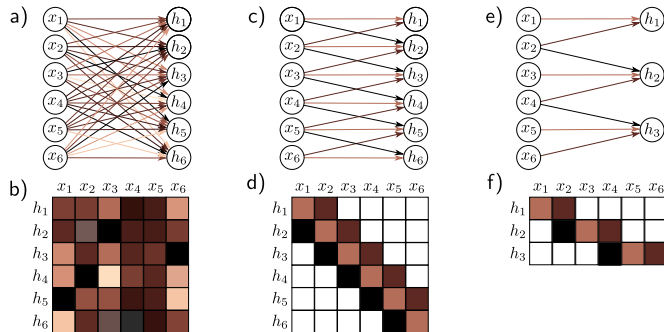


Figure 4: Fully connected matrices vs. convolution weight matrices.

Channels

- A single kernel detects only one pattern (e.g. vertical edge), which is *insufficient* for complex images.
- **Channels (Feature Maps):** To get a better idea, we apply multiple kernels simultaneously. Each unique kernel generates its own output channel.
- **Feature Specialization:** Kernels act as unique concept detectors:
 - *Early layers:* Detect basic elements (e.g., edges, color gradients).
 - *Deeper layers:* Detect complex concepts (e.g., textures, object parts).

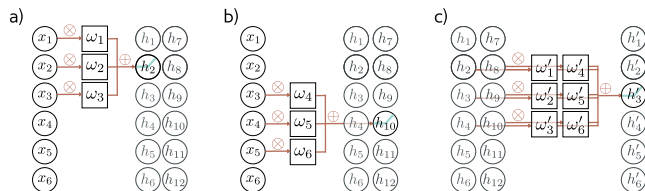


Figure 5: Multiple channels and stacked layers.

Receptive Fields

- **Receptive Field:** The region of the original input that feeds into a given hidden unit.
- Information from across the input is gradually integrated.
- Layer 1: Receptive field size 3.
- Layer 2: Receptive field size 5.
- Using stride > 1 or dilation rapidly increases the receptive field in deeper layers.

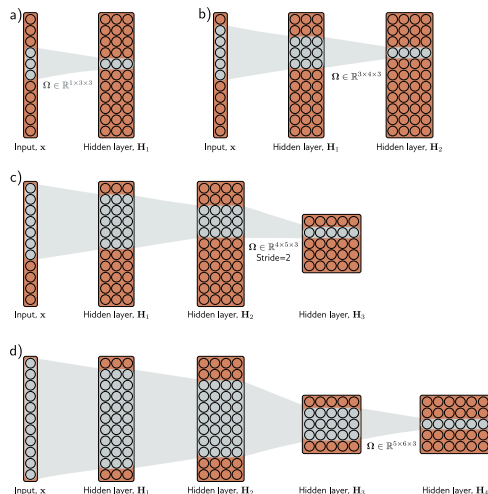


Figure 6: Growing receptive fields with depth.

Example: MNIST-1D

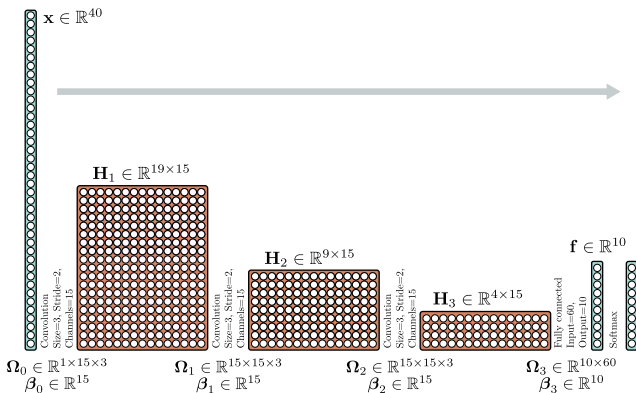


Figure 7: 1D CNN for MNIST-1D classification.

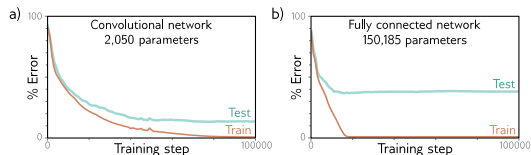


Figure 8: CNN vs FC Test Error.

- Both networks fit the training data perfectly.
- The CNN generalizes much better.
- This is due to a superior **inductive bias** (translation equivariance).

Convolutional Networks for 2D Inputs

- The convolutional kernel is a 2D object (e.g., 3×3).
- The kernel translates both horizontally and vertically.
-

$$h_{23} = a \left[\beta \right. \\ \left. + \omega_{11}x_{12} + \omega_{12}x_{13} + \omega_{13}x_{14} \right. \\ \left. + \omega_{21}x_{22} + \omega_{22}x_{23} + \omega_{23}x_{24} \right. \\ \left. + \omega_{31}x_{32} + \omega_{32}x_{33} + \omega_{33}x_{34} \right]$$

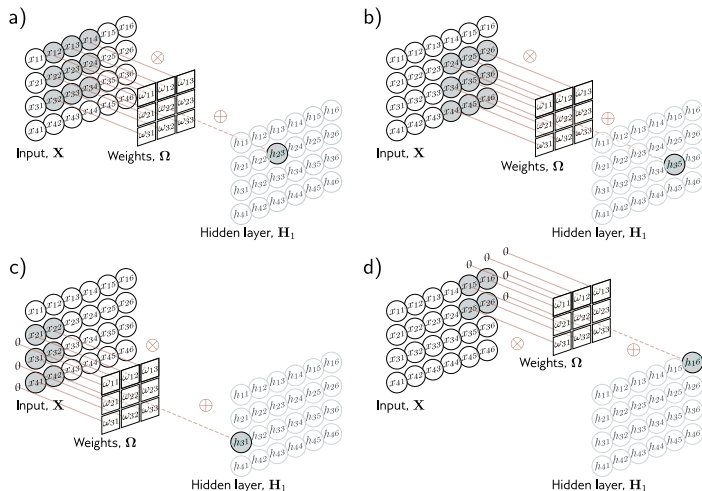


Figure 9: 2D Convolution on a grid.

Numeric Example: 2D Convolution

Valid Convolution: Sliding a 3×3 kernel over a 5×5 input gives a 3×3 output.

Input (A)

2	1	0	0	2
1	1	2	0	1
0	0	1	1	1
1	2	0	0	2
0	1	1	0	0

Kernel (K)

1	0	-1
1	0	-1
1	0	-1

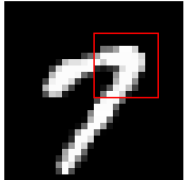
Output

0	1	-1
-1	2	-1
-1	2	-1

The diagram illustrates the convolution operation. The input matrix (A) is a 5x5 grid. A 3x3 kernel (K) is applied to a 3x3 region of the input, starting from the top-left. The output is a 3x3 grid. The value 0 in the top-left of the output is highlighted with a solid blue border, and the value 2 in the middle-right is highlighted with a dashed blue border.

Convolution over Images

1. Full MNIST Image
(Red box = Zoomed Area)



2. Zoomed Pixel Values
(0-99 Intensity)

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	44	46	62	98	98	82	0	0	0
66	97	97	97	97	97	83	0	0	0
98	78	97	97	97	97	87	0	0	0
15	7	15	25	86	97	97	71	0	0
0	0	0	58	97	97	85	7	0	0
0	0	0	69	97	97	54	0	0	0
0	6	34	93	97	75	26	0	0	0
0	46	97	97	81	9	0	0	0	0

3. Edge
Filter

1	1	1
1	-8	1
1	1	1

*

=

4. Output Feature Map
(Edge Highlights)

90	152	206	258	278	180	82	0
-46	29	-61	-333	-313	-261	262	83
-250	-158	-85	-33	-13	-135	-301	170
-132	-263	-165	-83	-11	-50	-251	241
247	242	347	-23	-23	-138	-195	165
37	174	-75	-150	-66	-160	251	78
40	260	-76	-93	-148	-45	172	7
129	136	-172	-158	-139	27	80	0

Sliding Window Result: (Input Region $\times$ Filter) = -165

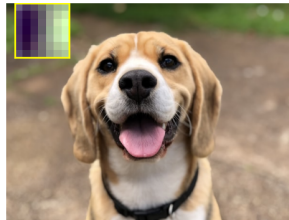
Feature Maps



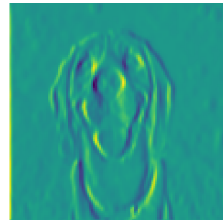
Original Image (Inset: Circle Kernel)



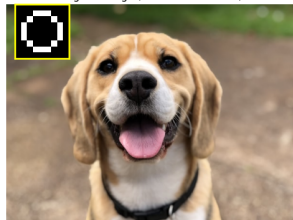
Circle Feature Map



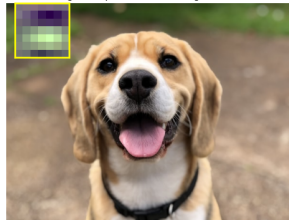
Original Input (Filter 22 Weights Inset)



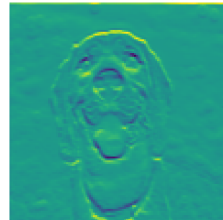
Filter #22 Feature Map



(a) Scenario 1: Custom Kernels



(b) Scenario 2: DenseNet121



2D Convolution on RGB Images

- RGB images have 3 channels.
- A 3×3 kernel applied to RGB has $3 \times 3 \times 3$ weights.
- Multiple output channels form a 3D tensor of hidden units.
- For C_i input and C_o output channels, $K \times K$ kernel uses $C_i \times C_o \times K \times K$ weights.

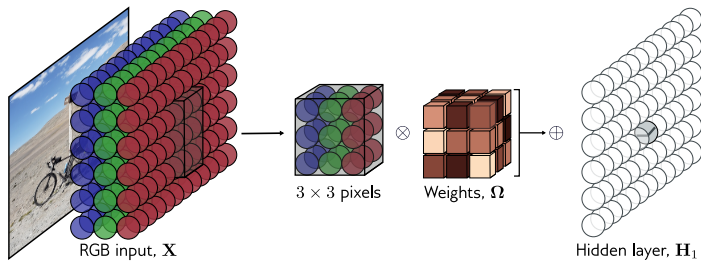


Figure 10: 2D Convolution on RGB input tensor.

Downsampling

Downsampling: Reduces the dimension of feature maps. Increases the receptive field, reduces computational cost and memory, extracts higher-level semantic features.

Methods:

- **Strided Convolution:** Evaluate output at every 2nd position.
- **Max Pooling:** Retains max of 2×2 block (induces local translation invariance).
- **Mean Pooling:** Averages inputs over the block.

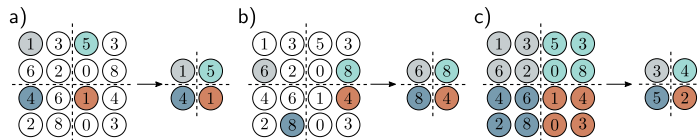


Figure 11: Methods for scaling down representation size.

Upsampling

Upsampling: Increases the dimension of feature maps. maps low-resolution, high-level features back to the original image resolution, which is required for dense, pixel-wise predictions (like semantic segmentation).

Methods:

- Duplicate: Duplicate each input four times.
- **Max Unpooling:** Redistribute value to the location where the max previously originated.
- Bilinear Interpolation.

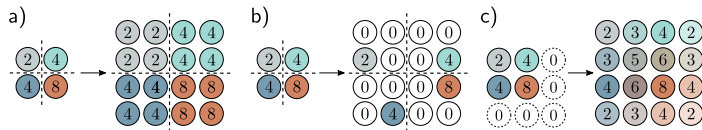


Figure 12: Methods for scaling up representation size.

Transposed Convolution

- A learnable form of upsampling.
- Downsampling with stride 2 maps 2 inputs to 1 output.
- **Transposed Convolution** maps 1 input to multiple outputs.
- Weight matrix associated with upsampling is the *transpose* of the downsampling matrix.

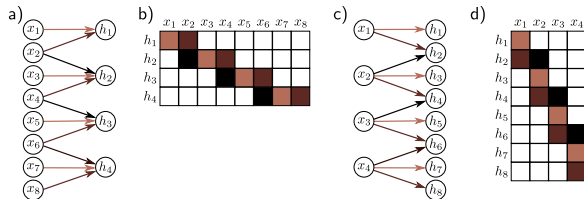


Figure 13: Transposed Convolution operations.

Changing the Number of Channels (1×1 Convolution)

- Sometimes we must alter channel count *without* changing spatial dimensions.
- Apply a 1×1 Convolution.
- Takes a weighted sum across the channel dimension at the exact same spatial pixel.

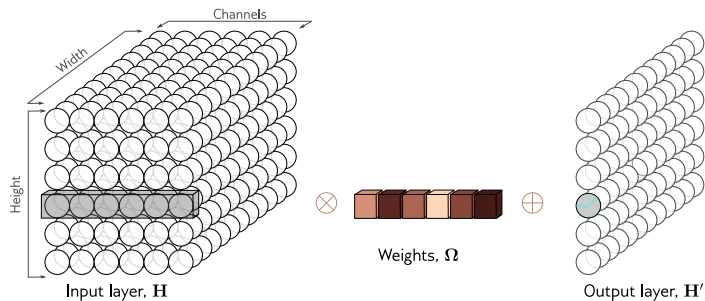


Figure 14: 1×1 Convolution mechanism.

Application 1: Image Classification (ImageNet)

- **ImageNet Challenge:** 1.2M training images, 1000 categories.
- Highly challenging due to visual variation: rigidity, clutter, size, shape, occlusion.
- Replaced traditional CV pipelines with end-to-end deep CNNs.



Figure 15: Diverse attributes: rigidity (monkey < canoe), number of instances in image (lizard < strawberry), clutter (compass < drum), size (candle < spiderweb), texture (screwdriver < leopard), distinctiveness of color (mug < red wine), and distinctiveness of shape (headland < bell).

Image Classification Architectures: AlexNet

- **AlexNet (2012):** First CNN to dominate ImageNet.
- 5 Conv Layers, 3 FC layers ($\sim 60\text{M}$ parameters).
- **Performance:** Achieved 83.6% top-5 accuracy and 62% top-1 accuracy.
- Ignited the modern era of Deep Learning.

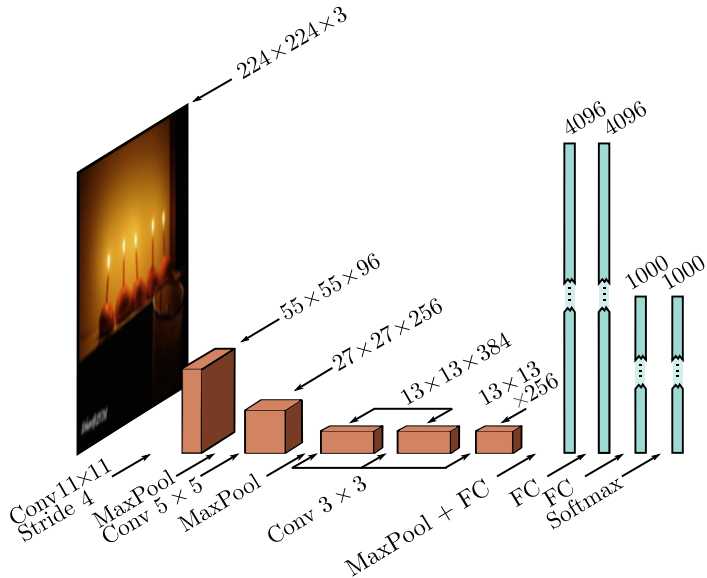


Image Classification Architectures: VGG

- **VGG Network (2014):** Standardized CNN design.
- Repeated sequence of 3×3 conv layers and 2×2 max pooling.
- Deep architecture: 19 hidden layers ($\sim 144\text{M}$ parameters).
- **Performance:** Achieved a considerably better 93.2% top-5 accuracy and 76.3% top-1 accuracy.

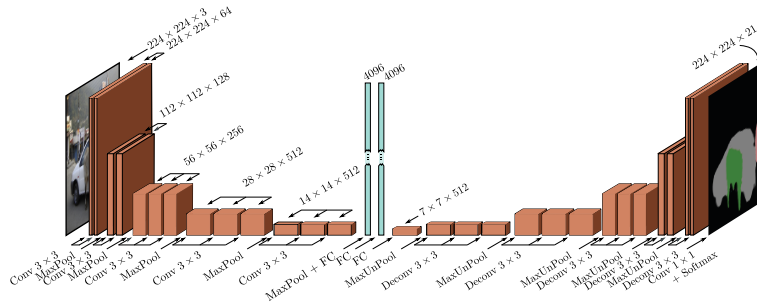


Figure 17: VGG network architecture.

Clarification: Detection vs. Segmentation

What is the difference?

Object Detection:

- Goal: Find individual objects, draw **bounding boxes** around them, and classify each box.
- Does not care about the exact shape or contours of the object.
- Output: $(x, y, w, h, \text{class_id}, \text{confidence})$.

Semantic Segmentation:

- Goal: Assign a class label to **every single pixel** in the image.
- Provides exact contours and boundaries of objects/regions (e.g., road, sky, car).
- Typically does *not* distinguish between two overlapping instances of the same class (e.g., two cars touching are segmented as one giant "car" blob).

Application 2: Object Detection (YOLO)

- **Goal:** Identify and localize multiple objects using bounding boxes.
- **YOLO (You Only Look Once):**
 - The input image is reshaped to 448×448 and divided into a regular 7×7 grid.
 - The system predicts the most likely class at each grid cell.
 - It also predicts two bounding boxes per cell, and a confidence value (thickness of line).
 - During inference, the most likely bounding boxes are retained, and boxes with lower confidence values for the same object are suppressed.

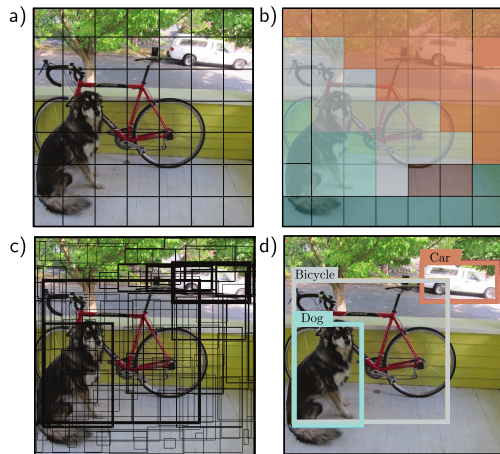


Figure 18: YOLO object detection grid and predictions.

Application 3: Semantic Segmentation

- **Goal:** Assign a class label to every individual pixel.
- Requires an **Encoder-Decoder** structure (hourglass).
- Encoder extracts contextual info.
- Decoder uses Max Unpooling and Transposed Convolutions to recover spatial resolution.

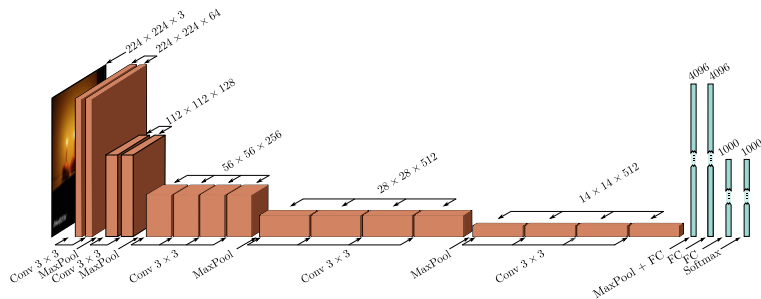


Figure 19: Encoder-Decoder architecture for segmentation.

Semantic Segmentation Results

- The network outputs 21 probability maps (one for each class).
- A greedy heuristic combines these maps into the final prediction.
- Encourages connectedness and spatial proximity of identified classes.

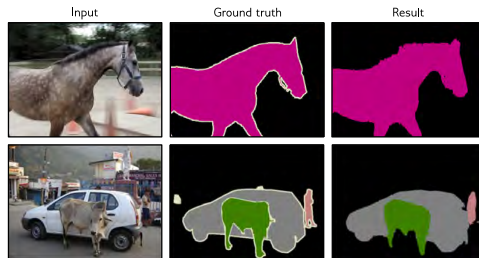


Figure 20: Semantic segmentation input, truth, and result.

Origins of Convolution

Historical Sidenote: Math & Engineering

Convolution is not an invention of Deep Learning. It is a fundamental mathematical operation originally used in engineering, physics, and signal processing.

- In continuous mathematics, the convolution of two functions f and g represents how the shape of one is modified by the other: $(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$.
- It has been used for decades in electrical engineering (filter design), acoustics, probability, and image processing (edge detection, blurring).
- Deep Learning adopts this operation, but instead of using hand-crafted filters, we let the network **learn** the kernel weights!

Explore CNNs interactively in your browser:

`https://poloclub.github.io/cnn-explainer/`

(Great for visualizing how kernels, ReLU, and pooling work in real-time!)

Summary

- **CNNs reduce parameter counts** via weight sharing and sparsity compared to fully connected models, mitigating overfitting on high-dimensional data like images.
- **Invariance & Equivariance:** Convolutions exhibit translation equivariance, serving as a powerful inductive bias for computer vision.
- **Receptive Field:** Hierarchical stacking of convolutions and pooling increases the span of inputs each deep node can "see."
- **Spatial manipulation:** Downsampling builds contextual features; upsampling recovers spatial structure for dense predictions.
- **Applications:** CNNs are foundational to modern Image Classification, Object Detection, and Semantic Segmentation tasks.